

STORE INTELLIGENCE PLATFORM

Business Requirements Document (BRD)

AI-Powered Retail Analytics and Store Intelligence System

Purpose

Define a production-grade system that converts raw CCTV footage into real-time business intelligence through detection, tracking, re-identification, event streaming, analytics, APIs, and live dashboards.

North Star Metric

Offline Store Conversion Rate = $\text{Converted Visitors} / \text{Total Unique Visitors}$. Every design decision should improve this metric or make it actionable in operations.

Document Focus

Business context, functional requirements, non-functional requirements, architecture, event model, API surface, data model, security, observability, deployment, testing, risks, and roadmap.

Prepared for a real-world AI + backend + systems engineering challenge.

Table of Contents

High-level structure of the BRD

- 1. Executive Summary
- 2. Business Context
- 3. Product Vision
- 4. Goals and Success Metrics
- 5. Functional Requirements
- 6. Non-Functional Requirements
- 7. System Architecture
- 8. Event-Driven Design
- 9. Data Modeling
- 10. API Design
- 11. Security Architecture
- 12. Observability and Reliability
- 13. AI/ML Architecture
- 14. Edge Cases and Operational Rules
- 15. Scalability Strategy
- 16. Deployment Strategy
- 17. Testing Strategy
- 18. Dashboard Design
- 19. Risk Analysis
- 20. Implementation Roadmap
- 21. Design Decisions
- 22. Success Criteria and Acceptance
- 23. Future Enhancements
- Appendices

1. Executive Summary

Why this platform exists and what success means

The Store Intelligence Platform (SIP) is a production-grade AI system that transforms raw CCTV footage from retail stores into real-time operational intelligence. It combines computer vision, tracking, re-identification, event streaming, analytics, REST APIs, and live dashboards into one end-to-end platform.

The challenge is not only to detect people in video. The real objective is to build a system that can answer business questions such as: how many visitors entered, how many converted, where customers dwell, where queues form, which stores underperform, and what anomalies are happening right now.

Business Outcome
Replace guesswork in offline retail with measurable, actionable, real-time intelligence that improves conversion, reduces queue frustration, optimizes staff allocation, and exposes store-level performance with e-commerce-like precision.

Success Metric	Target	Why It Matters
Visitor detection recall	>95%	Footfall must be trustworthy
Re-ID / re-entry accuracy	>85%	Avoid double counting and inflation
API p99 latency	<200 ms	Usable in real time
Event-to-dashboard delay	<1 second	Live operational awareness
Platform uptime	99.9%	Retail operations depend on it

2. Business Context

Retail analytics gap, stakeholders, and opportunity

Physical retail suffers from an analytics gap: online businesses track every click, scroll, and conversion, while brick-and-mortar stores often have only rough footfall counts and POS totals. This system closes that gap by converting camera footage into store intelligence.

Stakeholder	Need
CEO / COO	Revenue visibility, store benchmarking, strategic decision support
Store Manager	Live queue monitoring, staffing alerts, daily performance
Operations Team	Zone usage, layout inefficiencies, anomaly response
Data / AI Team	Model performance, drift monitoring, evaluation artifacts
Security / Compliance	Privacy, access control, retention, auditability

The platform should support the full customer journey inside a store:

- Entry: detect the moment a person crosses the threshold.
- Browse: track movement through product zones.
- Engage: measure dwell time in high-interest areas.
- Queue: detect wait-line formation and queue growth.
- Purchase: correlate billing activity with POS transactions.
- Exit: close the session and record visit duration.

3. Product Vision

Mission, differentiation, and long-term platform direction

Mission

Give physical retailers the same operational intelligence that digital platforms already enjoy by turning existing CCTV infrastructure into an analytics network.

The platform should differentiate itself in four ways:

- Real-time, not batch-only analytics.
- Session-level identity stitching and re-entry handling.
- Operationally useful metrics, not just raw detections.
- Production-grade architecture, observability, and resilience.

Capability	Traditional CCTV	SIP
Visitor count	Manual or approximate	Session-aware and deduplicated
Queue monitoring	Visual only	Measured depth and abandonment
Conversion measurement	POS-only	POS plus visitor correlation
Heatmaps	None	Zone-level live intelligence
Anomaly detection	Manual observation	Automated and alerted

4. Goals and Success Criteria

What the platform must achieve

Goal Type	Goal
Business	Improve conversion visibility and reduce friction in physical stores
AI	Detect, track, and re-identify people reliably in retail footage
Backend	Expose low-latency APIs and safe ingestion for events
Production	Support secure, observable, containerized deployment

Key operational targets:

- Support at least 40 stores in the first deployment wave.
- Allow horizontal scale to 400, 4,000, and 10,000+ stores.
- Keep event latency under 500 ms from frame to stored event.
- Handle zero-traffic periods, crowding, and camera overlap without crashing.

5. Functional Requirements

Core system behaviors by functional area

Area	Requirement	Priority
Detection	Detect every person in each CCTV clip and output structured events	P0
Tracking	Maintain identity across frames and handle occlusion	P0
Re-ID	Detect re-entry and stitch sessions across cameras	P0
Staff filter	Classify staff and exclude them from customer metrics	P0
Zones	Emit zone entry, exit, and dwell events	P0
Queue analytics	Track queue depth, wait time, and abandonment	P0
Conversion	Correlate POS transactions to sessions	P0
Dashboard	Show live metrics and anomalies	P1

Event catalogue should include ENTRY, EXIT, ZONE_ENTER, ZONE_EXIT, ZONE_DWELL, BILLING_QUEUE_JOIN, BILLING_QUEUE_ABANDON, and REENTRY.

Each event should contain a globally unique event_id, store_id, camera_id, visitor_id, timestamp, event_type, confidence, and metadata needed for analytics.

6. Non-Functional Requirements

Performance, security, resilience, and maintainability

Dimension	Requirement
Availability	99.9% uptime with graceful degradation
Latency	p99 API latency below 200 ms
Throughput	Support thousands of events per second
Security	TLS, JWT, HMAC signing, RBAC, audit logging
Reliability	Idempotent ingest and retry-safe processing
Maintainability	Containerized deployment and clear docs
Observability	Structured logs, metrics, tracing, health checks

- No raw stack traces in API responses.
- Database downtime should return a structured 503.
- Empty stores and zero-purchase stores must still return valid JSON.
- Event deduplication must be idempotent by event_id.

7. System Architecture

End-to-end pipeline and component responsibilities

Reference Architecture

CCTV -> Detection (YOLOv11) -> Tracking (ByteTrack) -> Re-ID (OSNet) -> Event Stream (Kafka) -> Processing Services -> PostgreSQL/TimescaleDB/Redis -> FastAPI -> React Dashboard

Layer	Component	Role
Edge	Detection service	Run inference close to the video source
Edge	Tracking service	Assign and maintain track IDs
Identity	Re-ID service	Link re-entry and cross-camera visits
Streaming	Kafka	Durable event transport
Processing	Session / metrics / anomaly workers	Convert events into intelligence
Storage	PostgreSQL + TimescaleDB + Redis	Persist and cache operational data
API	FastAPI	Serve analytics and ingest events
UI	React dashboard	Show live operational intelligence

8. Event-Driven Design

Schema, topics, idempotency, and retry strategy

Kafka Topic	Use
sip.detections.raw	Raw detection events from the edge
sip.tracking.updates	Track state changes
sip.reid.events	Session stitching and re-entry
sip.sessions.lifecycle	Session start/end and conversion correlation
sip.queue.metrics	Live queue depth and abandonment
sip.anomalies	Detected operational anomalies

Idempotency Strategy

Deduplicate by event_id, use idempotent Kafka producers, upsert state in the database, and keep a short TTL cache for recently processed event IDs.

9. Data Modeling

Core tables and analytics-ready storage

Table	Purpose
stores	Retail store master data
cameras	Camera metadata and coverage
zones	Named zones and geometry definitions
events	Raw event stream persisted for analytics
sessions	Visitor sessions and conversion linkage
transactions	POS transaction records
anomalies	Detected alerts and status
audit_logs	System audit trail

Recommended storage split:

- PostgreSQL for relational metadata, sessions, and transactions.
- TimescaleDB hypertables for time-series events and metrics.
- Redis for live session state, hot metrics, and dedupe TTL.

10. API Design

Operational API surface exposed to the dashboard and external consumers

Endpoint	Purpose
POST /events/ingest	Accept up to 500 events, validate, dedupe, and store
GET /stores/{id}/metrics	Return live metrics snapshot
GET /stores/{id}/funnel	Return conversion funnel counts and drop-off
GET /stores/{id}/heatmap	Return zone activity and dwell heatmap
GET /stores/{id}/anomalies	Return active anomalies and suggested actions
GET /health	Return system and dependency health

API Design Principles

Validate aggressively, fail gracefully, return machine-readable errors, support zero-data responses, and keep the API fast enough for live operations.

11. Security Architecture

Authentication, authorization, integrity, and privacy

- JWT authentication for users and service clients.
- HMAC signatures for event ingestion integrity.
- RBAC for dashboards, admin tools, and API consumers.
- TLS 1.3 in transit and AES-256 at rest.
- No face images or biometric artifacts should be persisted.
- Structured audit logging for all access and mutation actions.

Threat	Mitigation
Replay attacks	Timestamped HMAC signatures and short replay window
SQL injection	Parameterized queries and ORM-based access
Privilege escalation	Strict RBAC and scoped tokens
Data exfiltration	Audit logs, role separation, least privilege

12. Observability and Reliability

Make the system inspectable and operable

Signal	Tooling	Examples
Logs	Structured JSON logs	trace_id, store_id, endpoint, latency_ms, status_code
Metrics	Prometheus	Ingest rate, lag, queue depth, active sessions
Tracing	OpenTelemetry	Request spans across services
Alerting	Grafana / Alertmanager	Feed lag, API failures, queue spikes

Health Signals

The /health endpoint should include service status, last event timestamp per store, stale feed warnings, and downstream dependency health.

13. AI and ML Architecture

Model choices, calibration, and evaluation

Component	Recommended Choice	Reason
Detection	YOLOv11	Strong speed-accuracy balance
Tracking	ByteTrack	Robust under occlusion and crowded scenes
Re-ID	OSNet	Practical person re-identification embeddings
Anomaly detection	EWMA + z-score + rules	Simple, explainable, and effective
Staff classification	Vision + behavior heuristics	Better retail-specific precision

- Measure detection precision, recall, MOTA, and IDF1.
- Calibrate confidence thresholds so low-confidence detections are not silently dropped.
- Track model drift and poor-scene performance over time.

14. Edge Cases and Operational Rules

How the system behaves when reality is messy

Edge Case	Expected Behavior
Group entry	Count each person individually, not as a single group
Re-entry	Same physical person returns as REENTRY, not a new customer
Camera overlap	Avoid double counting across overlapping fields of view
Empty store	Return zeros, not nulls or errors
Staff movement	Exclude staff from customer metrics
Crowded billing	Track queue depth and abandonment
Low light / glare	Degrade gracefully and keep confidence visible

15. Scalability Strategy

How the system grows from pilot to enterprise

Scale Tier	Expected Load	Architecture Implication
Pilot	40 stores	Single-region deployment with modest partitioning
Growth	400 stores	Scale consumers, DB replicas, and cache
Enterprise	4,000 stores	Sharding and stronger partition strategy
Global	10,000+ stores	Multi-region and edge-heavy processing

- Partition Kafka topics by store_id to preserve locality.
- Use cache-first access for hot operational metrics.
- Separate write-heavy ingestion from read-heavy analytics APIs.
- Use background workers for aggregation instead of on-demand computation.

16. Deployment Strategy

How the system runs locally and in production

Environment	Strategy
Local	Docker Compose with one-command startup
Staging	Mirrors production services and datasets
Production	Containerized deployment with CI/CD
Scaling	Horizontal scaling for API and consumer services

Production Readiness

The README should explain how to run detection against clips, ingest generated events, and verify metrics with no manual setup beyond git clone and docker compose up.

17. Testing Strategy

Prove correctness, robustness, and operability

- Unit tests for event parsing, metrics, and anomaly detection.
- Integration tests for API endpoints and database persistence.
- Pipeline tests for detection output schema and deduplication.
- Edge-case tests for empty stores, zero purchases, and re-entry.
- Load tests for event ingestion and dashboard-read endpoints.
- Coverage target above 70% with emphasis on business logic.

Test Type	Goal
Schema validation	All emitted events must conform to contract
Idempotency	Repeated ingest must not duplicate metrics
Resilience	Service degrades gracefully when a dependency fails
Performance	Meet latency and throughput targets under load

18. Dashboard Design

What the live operational UI should show

Widget	Purpose
Live visitor count	Show current active traffic
Queue depth	Show current load at billing
Conversion rate	Track store effectiveness
Zone heatmap	Identify hot and dead zones
Anomalies panel	Surface operational issues quickly

Dashboard Principle

Show one metric changing in real time so reviewers can see the pipeline and the API are genuinely connected.

19. Risk Analysis

What could fail and how to reduce the risk

Risk	Mitigation
Occlusion	Use robust tracking and conservative confidence handling
Re-ID errors	Tune thresholds and review ambiguous matches
Camera failure	Use health warnings and stale-feed detection
Database pressure	Use caching, batching, and time-series storage
Event duplication	Idempotency keys and dedupe cache
Privacy concerns	No raw face storage, strict anonymization

20. Implementation Roadmap

Phase plan from prototype to production

Phase	Focus
Phase 1	Detection pipeline and event schema
Phase 2	Event ingestion, metrics, and funnel APIs
Phase 3	Anomaly detection and observability
Phase 4	Dashboard and live streaming
Phase 5	Hardening, scale testing, and documentation

- Build the simplest correct version first.
- Validate schema compliance early.
- Wire metrics into the dashboard before polishing visuals.
- Write docs and tests as part of the implementation, not after.

21. Design Decisions

The most important trade-offs to document

Decision	Recommended Choice	Reason
Detector	YOLOv11	Best balance of speed and accuracy
Tracker	ByteTrack	Reliable in crowded retail footage
Re-ID	OSNet	Practical and proven for person matching
Storage	PostgreSQL + TimescaleDB	Good mix of relational and time-series workloads
API	FastAPI	Fast to build and production-friendly
Dashboard	React + WebSocket	Real-time UI with strong ecosystem

22. Success Criteria and Acceptance

What a passing submission should prove

- docker compose up starts the platform without manual setup.
- The detection pipeline produces valid structured events.
- POST /events/ingest accepts batches without 5xx errors.
- GET /stores/{id}/metrics returns valid JSON and real metrics.
- DESIGN.md and CHOICES.md explain architecture and trade-offs.
- The live dashboard reflects real event flow.

Acceptance Gate

A strong submission must not only work; it must be explainable, reproducible, and operationally credible.

23. Future Enhancements

What to build after the core system works

- Predictive staffing recommendations.
- Zone-level demand forecasting.
- Customer journey intelligence across multiple visits.
- GenAI store assistant for ops queries.
- Digital twin simulations for layout changes.
- Cross-store benchmark intelligence.

Appendix A. Key Event Fields

Suggested event contract for implementation

Field	Meaning
event_id	Unique UUID for every event
store_id	Store identifier from store metadata
camera_id	Camera that produced the event
visitor_id	Anonymized visitor session token
event_type	ENTRY, EXIT, ZONE_ENTER, ZONE_DWELL, etc.
timestamp	ISO-8601 UTC event time
confidence	Detection confidence value
metadata	Queue depth, session sequence, zone details

This BRD is designed to be the foundation for DESIGN.md, CHOICES.md, the implementation plan, and the final engineering submission.